

Command Line Interface Cheat Sheet

Windows CMD · PowerShell · Bash (Linux and macOS) · Git

How to read this sheet. Each row shows the same task in all three shells. Commands are in monospace; *grey italics* explain flags or name an alias for the same command.  needs administrator, root or sudo.  destructive; check before running. (Linux) or (macOS) marks a command that exists only on that platform. ★ marks the everyday essentials. Click any command to copy it.

Everyday Essentials

The commands most people use every day, in short form. Click a task name for the full entry with more options and notes.

Task	CMD	PowerShell	Bash
★ <u>Change directory</u>	cd C:\Users cd ../Desktop	cd "C:\Program Files" Set-Location ~\Desktop	cd /home/user/docs cd ~/Desktop
★ <u>List files</u>	dir dir /A <i>(include hidden)</i>	Get-ChildItem <i>(alias: ls, dir, gci)</i> ls -Force <i>(include hidden)</i>	ls ls -la <i>(long, incl. hidden)</i>
★ <u>Copy files</u>	copy file.txt C:\Backup copy *.txt D:\Backup	Copy-Item file.txt ~\Backup <i>(alias: cp)</i> Copy-Item *.txt -Destination C:\Backup	cp file.txt ~/Backup cp -i file.txt ~/Backup <i>(prompt before overwrite)</i>
★ <u>Move or rename</u>	move file.txt C:\New ren old.txt new.txt	Move-Item file.txt C:\New <i>(alias: mv)</i> Rename-Item old.txt new.txt	mv file.txt ~/new/ mv old.txt new.txt
★ <u>Delete files</u>	del oldfile.txt del *.tmp	Remove-Item oldfile.txt <i>(alias: rm, del)</i> Remove-Item *.tmp -WhatIf <i>(dry run)</i>	rm oldfile.txt rm -i file.txt <i>(confirm)</i>
★ <u>View file contents</u>	type readme.txt	Get-Content readme.txt <i>(alias: cat, type, gc)</i>	cat readme.txt less readme.txt <i>(paginated; q to quit)</i>
★ <u>Find files</u>	dir /S /B *.txt where /R C:\Projects *.log	Get-ChildItem -Recurse -Filter *.txt gci -r -Include *.log,*.txt	find . -name "*.txt" find /home -type f -name "*.log"
★ <u>Search text in files</u>	findstr "error" log.txt findstr /S /I "text" *.* <i>(recursive, ignore case)</i>	Select-String -Path log.txt -Pattern error <i>(alias: sls)</i> sls error *.log -CaseSensitive	grep "error" log.txt grep -ri "error" /var/log/ <i>(recursive, ignore case)</i>
★ <u>List processes</u>	tasklist tasklist /V <i>(verbose)</i>	Get-Process <i>(alias: ps, gps)</i> Get-Process -Name chrome	ps aux ps aux grep nginx
★ <u>Kill process</u>	taskkill /IM notepad.exe taskkill /PID 1234 /F <i>(force)</i>	Stop-Process -Name notepad <i>(alias: kill)</i> Stop-Process -Id 1234 -Force	kill 1234 <i>(SIGTERM, polite)</i> kill -9 1234 <i>(SIGKILL, force)</i>

Task	CMD	PowerShell	Bash
★ <u>Ping host</u>	ping google.com <i>(4 packets)</i> ping -n 10 192.168.1.1	Test-Connection google.com Test-Connection 8.8.8.8 -Count 5	ping google.com <i>(until Ctrl+C)</i> ping -c 5 google.com
★ <u>IP configuration</u>	ipconfig ipconfig /all <i>(MAC, DNS, DHCP)</i>	Get-NetIPConfiguration Get-NetIPAddress -AddressFamily IPv4	ip addr <i>(Linux; short form: ip a)</i> ifconfig <i>(macOS; deprecated on Linux)</i>
★ <u>Show environment variables</u>	set set PATH <i>(all starting with PATH)</i>	Get-ChildItem Env: <i>(or: dir env:)</i> \$env:PATH	printenv env
★ <u>Command history</u>	doskey /history F7 <i>(popup list)</i>	Get-History <i>(alias: h, history)</i> Get-Content (Get-PSReadLineOption).HistorySavePath <i>(all sessions)</i>	history history 20 <i>(last 20)</i>
★ <u>Get help</u>	dir /? help copy	Get-Help Get-Process -Examples Get-Help *-Service <i>(search by name)</i>	man ls ls --help

File and Directory Management

Task	CMD	PowerShell	Bash	Notes
★ <u>Change directory</u>	cd C:\Users cd ../Desktop cd /d D:\Data <i>(switch drive)</i> D: <i>(switch drive only)</i>	cd "C:\Program Files" Set-Location ~\Desktop cd - <i>(previous dir, PS 6+)</i>	cd /home/user/docs cd ~/Desktop cd - <i>(previous dir)</i> cd <i>(home)</i>	cd .. goes to the parent. ~ is the home directory in PowerShell and Bash. CMD needs /d to change drive and directory at once.
★ <u>List files</u>	dir dir /A <i>(include hidden)</i> dir /S <i>(recursive)</i> dir /B <i>(bare names)</i> dir /O-D <i>(newest first)</i>	Get-ChildItem <i>(alias: ls, dir, gci)</i> ls -Force <i>(include hidden)</i> Get-ChildItem -Recurse ls *.txt	ls ls -la <i>(long, incl. hidden)</i> ls -lh <i>(readable sizes)</i> ls -lt <i>(newest first)</i> ls -R <i>(recursive)</i>	
<u>Show current directory</u>	cd <i>(no arguments)</i> echo %CD%	Get-Location <i>(alias: pwd)</i> \$PWD	pwd	
<u>Create directory</u>	mkdir MyReports md Folder1\Folder2 <i>(nested is fine)</i>	mkdir ~/Projects/New New-Item -ItemType Directory MyReports	mkdir MyReports mkdir -p a/b/c <i>(nested)</i>	
<u>Remove directory</u>	rmdir OldFolder <i>(empty only)</i> rmdir /S /Q OldFolder <i>(with contents, no prompt)</i>	Remove-Item OldFolder -Recurse -Force Remove-Item OldFolder -Recurse -WhatIf <i>(preview)</i>	rmdir OldFolder <i>(empty only)</i> rm -r OldFolder rm -rf OldFolder <i>(no prompts)</i>	⚠ Recursive deletes bypass the Recycle Bin / Trash. There is no undo.
<u>Directory tree</u>	tree C:\Projects tree /F <i>(include files)</i>	tree /F <i>(same tree.com as CMD)</i> Get-ChildItem -Recurse -Name	tree ~/Projects tree -L 2 <i>(limit depth)</i>	tree is not preinstalled on most Linux distros or macOS: apt install tree, brew install tree.

Task	CMD	PowerShell	Bash	Notes
★ Copy files	copy file.txt C:\Backup copy *.txt D:\Backup	Copy-Item file.txt ~\Backup <i>(alias: cp)</i> Copy-Item *.txt -Destination C:\Backup	cp file.txt ~/Backup cp -i file.txt ~/Backup <i>(prompt before overwrite)</i>	
Copy directories	xcopy C:\Src D:\Dst /E /I robocopy C:\Src D:\Dst /E robocopy C:\Src D:\Dst /MIR <i>(mirror; deletes extras)</i>	Copy-Item folder -Recurse -Destination dest	cp -r folder/ ~/dest cp -a folder/ ~/dest <i>(preserve attributes)</i> rsync -av src/ dest/	robocopy and rsync are the robust choices for large copies: both can resume and mirror. ⚠ /MIR and rsync --delete remove files at the destination.
★ Move or rename	move file.txt C:\New ren old.txt new.txt	Move-Item file.txt C:\New <i>(alias: mv)</i> Rename-Item old.txt new.txt	mv file.txt ~/new/ mv old.txt new.txt	
★ Delete files	del oldfile.txt del *.tmp del /P file.txt <i>(prompt)</i>	Remove-Item oldfile.txt <i>(alias: rm, del)</i> Remove-Item *.tmp -WhatIf <i>(dry run)</i>	rm oldfile.txt rm -i file.txt <i>(confirm)</i> rm *.tmp	⚠ No Recycle Bin. In PowerShell, -WhatIf previews any destructive cmdlet.
★ View file contents	type readme.txt	Get-Content readme.txt <i>(alias: cat, type, gc)</i>	cat readme.txt less readme.txt <i>(paginated; q to quit)</i>	
Create empty file	type nul > notes.txt copy con notes.txt <i>(type text, then Ctrl+Z, Enter)</i>	New-Item -ItemType File notes.txt <i>(alias: ni)</i>	touch notes.txt > notes.txt	touch also updates the timestamp of an existing file. New-Item errors if the file exists unless you add -Force.
★ Find files	dir /S /B *.txt where /R C:\Projects *.log	Get-ChildItem -Recurse -Filter *.txt gci -r -Include *.log,*.txt gci -r Where LastWriteTime -gt (Get-Date).AddDays(-1)	find . -name "*.txt" find /home -type f -name "*.log" find . -mtime -1 <i>(modified in last day)</i> find . -size +100M	
Which command runs	where python	Get-Command python Get-Command ls <i>(shows what an alias points to)</i>	which python type ls <i>(shows aliases and builtins too)</i> command -v python	
Compare files	fc file1.txt file2.txt fc /B a.bin b.bin <i>(binary)</i>	Compare-Object (Get-Content a.txt) (Get-Content b.txt) <i>(alias: diff)</i>	diff file1.txt file2.txt diff -u a.txt b.txt <i>(unified format)</i>	PowerShell's diff alias is not Unix diff: it compares sets of objects and ignores line order unless you add -SyncWindow 0.
★ Search text in files	findstr "error" log.txt findstr /S /I "text" *.* <i>(recursive, ignore case)</i> findstr /N "text" file.txt <i>(line numbers)</i>	Select-String -Path log.txt -Pattern error <i>(alias: sls)</i> sls error *.log -CaseSensitive gci -r *.log sls error	grep "error" log.txt grep -ri "error" /var/log/ <i>(recursive, ignore case)</i> grep -n "text" file.txt <i>(line numbers)</i>	

Task	CMD	PowerShell	Bash	Notes
File permissions	icacls file.txt icacls file.txt /grant User:F	Get-Acl file.txt Set-Acl file.txt -AclObject \$acl icacls file.txt <i>(works here too)</i>	chmod 755 script.sh chmod +x script.sh chown user:group file.txt	🔒 Changing permissions on files you do not own needs admin or sudo. Windows uses ACLs; Unix uses rwx bits.
File attributes	attrib +r file.txt <i>(read-only)</i> attrib +h file.txt <i>(hidden)</i> attrib -r -h file.txt	Set-ItemProperty file.txt IsReadOnly \$true (Get-Item file.txt).Attributes	chattr +i file.txt <i>(immutable, Linux)</i> chattr -i file.txt chflags hidden file.txt <i>(macOS)</i>	Unix has no hidden attribute: files whose names start with a dot are hidden by convention.
Symbolic links	mklink link.txt C:\target.txt mklink /D linkdir C:\target	New-Item -ItemType SymbolicLink -Path link -Target target	ln -s target link ln -s /path/to/dir linkdir	🔒 Windows needs an admin prompt or Developer Mode to create symlinks. Bash argument order is target first, then link.
Compress / archive	tar -a -cf archive.zip folder <i>(Windows 10 1803+)</i> compact /C file.txt <i>(NTFS compression, not an archive)</i>	Compress-Archive -Path folder -DestinationPath archive.zip	tar -czvf archive.tar.gz folder/ zip -r archive.zip folder/ gzip file.txt <i>(replaces file with file.txt.gz)</i>	bsdtar ships with Windows 10 and later and works in both CMD and PowerShell. It reads .zip, .tar.gz, .7z and more.
Extract	tar -xf archive.zip tar -xf archive.tar.gz expand archive.cab C:\dest <i>(CAB only)</i>	Expand-Archive archive.zip -DestinationPath dest\	unzip archive.zip tar -xzf archive.tar.gz gunzip file.txt.gz	
File hash / checksum	certutil -hashfile file.iso SHA256	Get-FileHash file.iso <i>(SHA256 by default)</i> Get-FileHash file.iso -Algorithm MD5	sha256sum file.iso md5sum file.iso shasum -a 256 file.iso <i>(macOS)</i>	
Open with default app	start report.pdf start . <i>(Explorer here)</i> start https://example.com	Invoke-Item report.pdf <i>(alias: ii)</i> start . <i>(alias of Start-Process)</i>	xdg-open report.pdf <i>(Linux)</i> open report.pdf <i>(macOS)</i> open . <i>(Finder here, macOS)</i>	
Clear screen	cls	Clear-Host <i>(alias: cls, clear)</i> Ctrl+L	clear Ctrl+L	

Disk and Storage

Task	CMD	PowerShell	Bash	Notes
Free disk space	fsutil volume diskfree C: wmic logicaldisk get name,size,freespace <i>(wmic is deprecated)</i>	Get-PSDrive -PSProvider FileSystem Get-Volume Get-CimInstance Win32_LogicalDisk Select DeviceID,Size,FreeSpace	df -h df -h /home <i>(one filesystem)</i>	wmic is deprecated and removed from recent Windows 11 builds. Prefer Get-CimInstance.

Task	CMD	PowerShell	Bash	Notes
Disk and partition info	diskpart <i>(interactive: list disk, list volume)</i> wmic diskdrive get model,size <i>(deprecated)</i>	Get-Disk Get-Partition Get-Volume	lsblk <i>(Linux)</i> sudo fdisk -l <i>(Linux)</i> diskutil list <i>(macOS)</i>	
Folder size	dir /S C:\folder <i>(total at the end)</i>	(Get-ChildItem -Recurse folder Measure-Object Length -Sum).Sum / 1MB	du -sh folder/ du -h --max-depth=1 . sort -h <i>(Linux)</i> du -sh * sort -h	
Check disk	chkdsk C: chkdsk C: /F <i>(fix errors; system drive needs a reboot)</i>	Repair-Volume -DriveLetter C -Scan Repair-Volume -DriveLetter C -OfflineScanAndFix	sudo fsck /dev/sda1 <i>(unmount first)</i> diskutil verifyVolume / <i>(macOS)</i>	🔒 Requires admin or sudo. Never run fsck on a mounted filesystem.
Format drive	format D: /FS:NTFS /Q	Format-Volume -DriveLetter D -FileSystem NTFS	sudo mkfs.ext4 /dev/sdb1 diskutil eraseDisk APFS Name disk2 <i>(macOS)</i>	⚠️ Destroys everything on the volume. Double-check the drive letter or device name first.
Mount / map drives	net use Z: \\server\share <i>(map network drive)</i> net use Z: /delete mountvol <i>(list volume mount points)</i>	New-PSDrive -Name Z -PSProvider FileSystem -Root \\server\share -Persist Mount-DiskImage file.iso Dismount-DiskImage file.iso	sudo mount /dev/sdb1 /mnt/usb sudo umount /mnt/usb mount <i>(list mounts)</i>	

System and User Information

Task	CMD	PowerShell	Bash	Notes
Current user	whoami echo %USERNAME% whoami /groups	whoami \$env:USERNAME [Environment]::UserName	whoami echo \$USER id <i>(uid, gid and groups)</i>	
Hostname	hostname echo %COMPUTERNAME%	hostname \$env:COMPUTERNAME	hostname hostnamectl <i>(Linux with systemd)</i> scutil --get ComputerName <i>(macOS)</i>	
System information	systeminfo systeminfo findstr /B /C:"OS Name" /C:"OS Version"	Get-ComputerInfo Get-ComputerInfo Select OsName,OsVersion,CsTotalPhysicalMemory Get-CimInstance Win32_OperatingSystem	uname -a cat /etc/os-release <i>(Linux)</i> hostnamectl <i>(Linux)</i> sw_vers <i>(macOS)</i>	Get-ComputerInfo is slow (several seconds). Pick properties with Select-Object.
OS version	ver	[Environment]::OSVersion \$PSVersionTable.PSVersion <i>(PowerShell version)</i>	uname -r <i>(kernel)</i> lsb_release -a <i>(Debian/Ubuntu)</i> sw_vers -productVersion <i>(macOS)</i>	

Task	CMD	PowerShell	Bash	Notes
CPU	echo %NUMBER_OF_PROCESSORS% wmic cpu get name,numberofcores <i>(deprecated)</i>	Get-CimInstance Win32_Processor Select Name,NumberOfCores,NumberOfLogicalProcessors	lscpu <i>(Linux)</i> nproc <i>(Linux)</i> sysctl -n machdep.cpu.brand_string <i>(macOS)</i>	
Memory	systeminfo findstr /C:"Total Physical Memory" /C:"Available Physical Memory"	Get-CimInstance Win32_OperatingSystem Select TotalVisibleMemorySize,FreePhysicalMemory Get-CimInstance Win32_PhysicalMemory Select Capacity,Speed	free -h <i>(Linux)</i> cat /proc/meminfo <i>(Linux)</i> vm_stat <i>(macOS)</i> top <i>(any)</i>	
Uptime	systeminfo findstr /C:"System Boot Time" net statistics workstation findstr "since"	(Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime Get-Uptime <i>(PowerShell 6+ only)</i>	uptime uptime -p <i>(pretty, Linux)</i> uptime -s <i>(boot time, Linux)</i>	
Date and time	date /T time /T echo %DATE% %TIME%	Get-Date Get-Date -Format "yyyy-MM-dd HH:mm:ss" Get-Date -UFormat "%Y-%m-%d"	date date +"%Y-%m-%d %H:%M:%S" date -u <i>(UTC)</i>	Without /T, CMD's date and time commands prompt you to change the clock (🕒 admin).
Logged-in users	query user <i>(alias: quser)</i>	query user Get-CimInstance Win32_ComputerSystem Select UserName	who w <i>(who, plus what they are running)</i> last <i>(login history)</i>	

Processes, Services and Power

Task	CMD	PowerShell	Bash	Notes
★ List processes	tasklist tasklist /V <i>(verbose)</i> tasklist /FI "IMAGENAME eq chrome.exe"	Get-Process <i>(alias: ps, gps)</i> Get-Process -Name chrome Get-Process Sort-Object CPU -Descending Select -First 10	ps aux ps aux grep nginx top <i>(live)</i> htop <i>(live, nicer; needs install)</i>	
★ Kill process	taskkill /IM notepad.exe taskkill /PID 1234 /F <i>(force)</i> taskkill /F /IM chrome.exe /T <i>(with child processes)</i>	Stop-Process -Name notepad <i>(alias: kill)</i> Stop-Process -Id 1234 -Force	kill 1234 <i>(SIGTERM, polite)</i> kill -9 1234 <i>(SIGKILL, force)</i> pkill -f pattern killall firefox	⚠ Force-killing skips cleanup and can corrupt files being written. Try the polite form first.
Find process using a port	netstat -ano findstr :8080 <i>(PID in last column)</i>	Get-NetTCPConnection -LocalPort 8080 Select OwningProcess Get-Process -Id (Get-NetTCPConnection - LocalPort 8080).OwningProcess	sudo lsof -i :8080 sudo ss -ltnp grep 8080 <i>(Linux)</i>	

Task	CMD	PowerShell	Bash	Notes
Run in background	start /B program.exe <i>(same window)</i> start "" notepad.exe <i>(new window)</i>	Start-Process notepad Start-Job { long-task }; Get-Job; Receive-Job 1 command & <i>(PowerShell 6+)</i>	command & nohup command & <i>(survives logout)</i> jobs / fg / bg Ctrl+Z <i>(suspend the foreground job)</i>	
List services	sc query <i>(running)</i> sc query state= all net start <i>(running only)</i>	Get-Service Get-Service Where Status -eq Running Get-Service -Name Spooler	systemctl list-units --type=service <i>(Linux)</i> systemctl list-units --type=service -state=running launchctl list <i>(macOS)</i>	
Start / stop / restart service	sc start Spooler sc stop Spooler net stop Spooler && net start Spooler <i>(net start/stop also work)</i>	Start-Service Spooler Stop-Service Spooler Restart-Service Spooler	sudo systemctl start nginx sudo systemctl stop nginx sudo systemctl restart nginx sudo systemctl enable nginx <i>(start at boot)</i>	🔒 Requires admin or sudo. sc and Get-Service use the service name (Spooler), not the display name (Print Spooler).
Service status	sc query Spooler sc qc Spooler <i>(configuration)</i>	Get-Service Spooler (Get-Service Spooler).Status	systemctl status nginx systemctl is-active nginx journalctl -u nginx -f <i>(follow logs, Linux)</i>	
Scheduled tasks	schtasks /query schtasks /create /tn Backup /tr C:\backup.bat /sc daily /st 02:00 schtasks /delete /tn Backup	Get-ScheduledTask Get-ScheduledTask Backup Start-ScheduledTask Register-ScheduledTask <i>(see the Get-Help examples)</i>	crontab -l <i>(list)</i> crontab -e <i>(edit)</i> 0 2 * * * /path/backup.sh <i>(daily at 02:00)</i> launchctl <i>(macOS launchd)</i>	
Restart system	shutdown /r /t 0 shutdown /a <i>(abort a pending shutdown)</i>	Restart-Computer Restart-Computer -Force	sudo reboot sudo shutdown -r now sudo shutdown -r +5 <i>(in 5 minutes)</i>	🔒 Requires admin or sudo.
Shut down system	shutdown /s /t 0 shutdown /s /t 60 <i>(in 60 seconds)</i> shutdown /h <i>(hibernate)</i>	Stop-Computer Stop-Computer -Force	sudo shutdown -h now sudo poweroff sudo shutdown -c <i>(cancel, Linux)</i>	🔒 Requires admin or sudo.

Networking and Diagnostics

Task	CMD	PowerShell	Bash	Notes
★ Ping host	ping google.com <i>(4 packets)</i> ping -n 10 192.168.1.1 ping -t host <i>(until Ctrl+C)</i>	Test-Connection google.com Test-Connection 8.8.8.8 -Count 5 Test-Connection host -Quiet <i>(True/False)</i>	ping google.com <i>(until Ctrl+C)</i> ping -c 5 google.com	Windows ping stops after 4 packets by default. Unix ping runs until you stop it.

Task	CMD	PowerShell	Bash	Notes
★ IP configuration	ipconfig ipconfig /all (MAC, DNS, DHCP) ipconfig /release && ipconfig /renew	Get-NetIPConfiguration Get-NetIPAddress -AddressFamily IPv4	ip addr (Linux; short form: ip a) ifconfig (macOS; deprecated on Linux) hostname -I (Linux, addresses only)	
Public IP	curl ifconfig.me	Invoke-RestMethod ifconfig.me	curl ifconfig.me	
Connections and open ports	netstat -an (all) netstat -ano (with PIDs) netstat -b (with program names, 🗝 admin)	Get-NetTCPConnection Get-NetTCPConnection -State Listen	ss -tulpn (Linux, preferred) sudo netstat -tulpn (older Linux) sudo lsof -i -P (macOS and Linux)	netstat and ifconfig are deprecated on Linux in favour of ss and ip, but still work on macOS.
DNS lookup	nslookup google.com nslookup google.com 8.8.8.8 (ask a specific server) nslookup -type=MX google.com	Resolve-DnsName google.com Resolve-DnsName google.com -Type MX Resolve-DnsName google.com -Server 8.8.8.8	dig google.com dig +short google.com dig MX google.com host google.com	
Trace route	tracert google.com tracert -d google.com (skip DNS, faster) pathping google.com	Test-NetConnection google.com -TraceRoute	tracert google.com tracert google.com (Linux, no root needed) mtr google.com (live; needs install)	
Routing table	route print route add 192.168.2.0 mask 255.255.255.0 192.168.1.1	Get-NetRoute New-NetRoute -DestinationPrefix 192.168.2.0/24 -NextHop 192.168.1.1 -InterfaceIndex 12	ip route (Linux) sudo ip route add 192.168.2.0/24 via 192.168.1.1 netstat -rn (macOS)	🗝 Adding routes needs admin or sudo.
Flush DNS cache	ipconfig /flushdns ipconfig /displaydns	Clear-DnsClientCache Get-DnsClientCache	sudo resolvectl flush-caches (Linux, systemd-resolved) sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder (macOS)	
ARP table	arp -a	Get-NetNeighbor Get-NetNeighbor -State Reachable	ip neigh (Linux) arp -a (macOS and Linux)	
Download file	curl -O https://example.com/file.zip curl -L -o out.zip URL (follow redirects, custom name) certutil -urlcache -split -f URL file.zip (legacy)	Invoke-WebRequest URL -OutFile file.zip (alias: iwr) Invoke-RestMethod URL (parses JSON) (alias: irm) curl.exe -O URL	wget URL curl -O URL curl -L -o out.zip URL	In Windows PowerShell 5.1, curl is an alias of Invoke-WebRequest. Type curl.exe to get the real tool.
Test a port	curl -v telnet://host:443 telnet host 80 (optional feature, off by default)	Test-NetConnection host -Port 443 (alias: tnc) tnc host -Port 443 -InformationLevel Quiet	nc -zv host 443 curl -v telnet://host:443 (echo > /dev/tcp/host/443) && echo open (Bash only)	

Task	CMD	PowerShell	Bash	Notes
SSH and remote copy	ssh user@host scp file.txt user@host:/path/ ssh -i key.pem user@host	ssh user@host scp file.txt user@host:/path/ Enter-PSession -ComputerName host (WinRM, Windows to Windows)	ssh user@host scp file.txt user@host:/path/ rsync -avz src/ user@host:/dest/	The OpenSSH client is built into Windows 10 1809 and later.

Environment, History and Shell Basics

Task	CMD	PowerShell	Bash	Notes
★ Show environment variables	set setx PATH (all starting with PATH) echo %PATH%	Get-ChildItem Env: (or: dir env:) \$env:PATH \$env:PATH -split ';' (one per line)	printenv env echo \$PATH echo \$PATH tr ':' '\n' (one per line)	
Set environment variable	set MYVAR=value (this session) setx MYVAR value (permanent, new windows only) setx PATH "%PATH%;C:\tools" (Δ truncates at 1024 chars)	\$env:MYVAR = 'value' (this session) [Environment]::SetEnvironmentVariable('MYVAR', 'value', 'User') (permanent)	export MYVAR=value (this session) MYVAR=value command (one command only) add the export line to ~/.bashrc or ~/.zshrc (permanent)	Permanent changes only affect shells opened afterwards.
★ Command history	doskey /history F7 (popup list)	Get-History (alias: h, history) Get-Content (Get-PSReadLineOption).HistorySavePath (all sessions)	history history 20 (last 20) history grep ssh	
Re-run a command	F3 or ↑ (previous)	↑ (previous) Invoke-History 5 (alias: r 5)	!! (previous) !5 (history entry 5) !ssh (last command starting with ssh) sudo !! (repeat with sudo)	
Search history	F7 (dialog) type a prefix, then F8 (cycle matches)	Ctrl+R (reverse search) type a prefix, then F8 Get-History Where CommandLine -like '*git*'	Ctrl+R (reverse search) history grep pattern	
Aliases	doskey ll=dir /B \$* (this session) (persist via a startup script in the AutoRun registry key)	Set-Alias ll Get-ChildItem Get-Alias (list all) function gs { git status } (aliases cannot take arguments; use a function)	alias ll='ls -la' alias (list all) unalias ll	

Task	CMD	PowerShell	Bash	Notes
Shell profile / startup	HKCU\Software\Microsoft\Command Processor\AutoRun <i>(registry value)</i>	\$PROFILE <i>(path to your profile script)</i> notepad \$PROFILE . \$PROFILE <i>(reload)</i>	~/ .bashrc <i>(interactive Bash)</i> ~/ .bash_profile <i>(login shells)</i> ~/ .zshrc <i>(zsh; default on macOS)</i> source ~/ .bashrc <i>(reload)</i>	Put aliases, functions and PATH changes in the profile so they survive between sessions.
Exit code of last command	echo %ERRORLEVEL% if %ERRORLEVEL% neq 0 echo failed	\$? <i>(True or False)</i> \$LASTEXITCODE <i>(from the last native .exe)</i>	echo \$? <i>(0 means success)</i>	
Chain commands	a && b <i>(b only if a succeeds)</i> a b <i>(b only if a fails)</i> a & b <i>(both, regardless)</i>	a; b <i>(both, regardless)</i> a && b, a b <i>(PowerShell 7+ only)</i> <i>(-and / -or are boolean operators, not chaining)</i>	a && b a b a; b	
★ Get help	dir /? help copy	Get-Help Get-Process -Examples Get-Help *-Service <i>(search by name)</i> Get-Command -Noun Service Update-Help <i>(once,  admin)</i>	man ls ls --help tldr ls <i>(needs install)</i>	
Run as admin / root	runas /user:Administrator cmd <i>(or right-click the shell and choose Run as administrator)</i>	Start-Process powershell -Verb RunAs sudo command <i>(Windows 11 24H2+; enable in Settings > System > For developers)</i>	sudo command sudo -i <i>(root shell)</i> su - <i>(switch user)</i>	

Text Processing and Output

Task	CMD	PowerShell	Bash	Notes
Display text	echo Hello World echo %PATH% echo. <i>(blank line)</i>	Write-Output 'Hello' <i>(alias: echo, write)</i> Write-Host 'Text' -ForegroundColor Red <i>(console only, not pipeable)</i> "Value: \$var" <i>(double quotes expand variables)</i>	echo "Hello World" printf "%s\n" "text" echo -e "a\tb" <i>(interpret escapes)</i>	
View a page at a time	more file.txt dir /S more	Get-Content file.txt more Get-Content file.txt Out-Host -Paging	less file.txt <i>(q quits, / searches)</i> more file.txt cat file.txt less	
First lines of a file	more +5 file.txt <i>(skip the first 5 instead)</i> <i>(none; use PowerShell)</i>	Get-Content file.txt -Head 10 Get-Content file.txt Select-Object -First 10	head file.txt <i>(10 lines)</i> head -n 20 file.txt head -c 100 file.bin <i>(bytes)</i>	
Last lines of a file	<i>(none; use PowerShell)</i>	Get-Content file.txt -Tail 10 Get-Content log.txt -Wait <i>(follow, like tail -f)</i>	tail file.txt tail -n 20 file.txt tail -f log.txt <i>(follow)</i>	

Task	CMD	PowerShell	Bash	Notes
Count lines / words	find /c /v "" < file.txt <i>(lines)</i> type file.txt find /c "error" <i>(matching lines)</i>	Get-Content file.txt Measure-Object - Line -Word -Character (Get-Content file.txt).Count <i>(lines)</i>	wc -l file.txt <i>(lines)</i> wc -w file.txt <i>(words)</i> grep -c error file.txt <i>(matching lines)</i>	
Sort lines	sort file.txt sort /R file.txt <i>(reverse)</i> dir /B sort	Get-Content file.txt Sort-Object Sort-Object -Unique Get-Process Sort-Object CPU -Descending	sort file.txt sort -r (reverse), -n (numeric), -u <i>(unique)</i> sort -k2 file.txt <i>(by second field)</i>	
Unique lines	<i>(none; use PowerShell)</i>	Get-Content file.txt Sort-Object -Unique Get-Content file.txt Group-Object Sort Count -Desc <i>(with counts)</i>	sort -u file.txt sort file.txt uniq -c <i>(with counts)</i> uniq file.txt <i>(adjacent duplicates only)</i>	
Filter a stream	dir find "txt" tasklist findstr /I chrome	Get-Process Where-Object Name -like 'chrome*' <i>(alias: where, ?)</i> Get-Process Where CPU -gt 100 gc log.txt sls error	ps aux grep chrome grep -v debug log.txt <i>(invert match)</i>	PowerShell pipes objects, not text: filter on properties with Where-Object, then shape output with Select-Object or Format-Table.
Find and replace	<i>(none; use PowerShell)</i>	(Get-Content file.txt) -replace 'old','new' Set-Content file.txt 'text' -replace '(\d+)','[\$1]' <i>(regex by default)</i>	sed 's/old/new/g' file.txt <i>(prints result)</i> sed -i 's/old/new/g' file.txt <i>(in place, Linux)</i> sed -i '' 's/old/new/g' file.txt <i>(in place, macOS)</i>	The parentheses around Get-Content read the whole file first so PowerShell can safely overwrite it.
Extract columns / fields	for /f "tokens=2 delims=," %i in (file.csv) do @echo %i <i>(inside a .bat file use %%i)</i>	Import-Csv data.csv Select-Object Name,Email 'a,b,c' -split ',' Get-Content f.txt ForEach { (\$_ -split '\s+')[1] }	cut -d, -f2 file.csv awk '{print \$2}' file.txt awk -F: '{print \$1}' /etc/passwd	
Shape and export output	dir /B /O-D <i>(bare, newest first)</i> tasklist /FO CSV tasklist /FO CSV > procs.csv	Get-Process Select-Object Name,CPU Get-Process Format-Table -AutoSize Get-Process Format-List * Get-Process Export-Csv procs.csv - NoTypeInfoation Get-Process ConvertTo-Json Get-Service Out-GridView	column -t file.txt <i>(align columns)</i> ls -l awk '{print \$9, \$5}' jq . file.json <i>(needs install)</i>	Format-* cmdlets must be last in a pipeline; their output is for the screen only.
Clipboard	dir clip	Get-Process Set-Clipboard <i>(alias: scb)</i> Get-Clipboard <i>(alias: gcb)</i> ls clip	ls pbcopy (macOS), pbpaste ls xclip -selection clipboard <i>(Linux, X11)</i> ls wl-copy <i>(Linux, Wayland)</i>	

Redirection and Piping

Task	CMD	PowerShell	Bash	Notes
Redirect output (overwrite) >	dir > list.txt	Get-Process > processes.txt Get-Process Out-File -Encoding utf8 p.txt Get-Process Set-Content p.txt	ls -la > listing.txt	Windows PowerShell 5.1 writes UTF-16 with >. Use Out-File -Encoding utf8 for files other tools will read. PowerShell 7 defaults to UTF-8.
Redirect output (append) >>	echo text >> file.txt	Get-Date >> log.txt Add-Content log.txt 'line'	echo "new line" >> file.txt	
Redirect input <	sort < unsorted.txt	Get-Content input.txt Sort-Object <i>(PowerShell has no < operator)</i>	sort < unsorted.txt cat <<EOF ... EOF <i>(here-document)</i> cmd <<< "string" <i>(here-string)</i>	
Pipe	dir find "txt" type file.txt more	Get-Process Sort-Object CPU -Descending Select -First 5 Get-ChildItem Where Length -gt 1MB	ls -la grep txt cat file.txt sort uniq -c	CMD and Bash pipe text. PowerShell pipes objects; \$_ (or \$PSItem) is the current object inside a script block.
Redirect errors 2>	command 2> errors.txt command 2>&1 <i>(merge errors into output)</i>	command 2> errors.txt command 2>&1 \$ErrorActionPreference = 'Stop' <i>(make errors terminate)</i>	command 2> errors.txt command 2>&1	
Redirect everything	command > out.txt 2>&1	command *> all.txt command 3>&1 (warnings), 4>&1 (verbose), 6>&1 <i>(information)</i>	command &> out.txt command > out.txt 2>&1	
Suppress output	command > nul command > nul 2>&1	command Out-Null command > \$null \$null = command <i>(fastest)</i>	command > /dev/null command &> /dev/null command 2> /dev/null <i>(errors only)</i>	
Tee (file and screen)	<i>(none; use PowerShell)</i>	Get-Process Tee-Object -FilePath p.txt Get-Process Tee-Object -Variable procs Select Name	ls tee listing.txt ls tee -a listing.txt <i>(append)</i> echo x sudo tee /etc/file <i>(write as root)</i>	
Command substitution	for /f %i in ('date /T') do set today=%i	\$today = Get-Date -Format yyyy-MM-dd Write-Output "Today: \$(Get-Date)" Copy-Item a.txt "backup-\$(Get-Date -f yyyyMMdd).txt"	today=\$(date +%F) echo "Today: \$(date)" cp a.txt "backup-\$(date +%F).txt"	

Users and Groups

Task	CMD	PowerShell	Bash	Notes
List users	net user net user alice <i>(details)</i>	Get-LocalUser Get-LocalUser Where Enabled	cat /etc/passwd <i>(Linux)</i> getent passwd dscl . list /Users <i>(macOS)</i>	
Add user	net user alice * /add <i>(prompts for password)</i>	New-LocalUser alice -Password (Read-Host - AsSecureString) New-LocalUser alice -NoPassword	sudo adduser alice <i>(interactive, Debian/Ubuntu)</i> sudo useradd -m alice && sudo passwd alice	🔒 Requires admin or sudo. Avoid typing passwords on the command line: they end up in history.
Delete user	net user alice /delete	Remove-LocalUser alice	sudo userdel alice sudo userdel -r alice <i>(also remove home dir)</i>	🔒 Requires admin or sudo.
Change password	net user alice * <i>(prompts)</i>	Set-LocalUser alice -Password (Read-Host - AsSecureString)	passwd <i>(your own)</i> sudo passwd alice	
List groups	net localgroup net localgroup Administrators <i>(members)</i>	Get-LocalGroup Get-LocalGroupMember Administrators	groups <i>(current user)</i> getent group id alice	
Add user to group	net localgroup Administrators alice /add	Add-LocalGroupMember -Group Administrators -Member alice	sudo usermod -aG sudo alice <i>(Debian/Ubuntu)</i> sudo usermod -aG wheel alice <i>(Fedora/RHEL)</i> <i>(log out and back in to apply)</i>	🔒 Requires admin or sudo. ⚠️ Forgetting -a in usermod removes the user from all other groups.

Package Management

Task	CMD	PowerShell	Bash	Notes
List installed	winget list	winget list choco list <i>(Chocolatey)</i> Get-Module -ListAvailable <i>(PowerShell modules)</i>	apt list --installed <i>(Debian/Ubuntu)</i> dnf list installed <i>(Fedora/RHEL)</i> brew list <i>(macOS)</i> pacman -Q <i>(Arch)</i>	wmic product get name is deprecated and very slow; use winget instead.
Search	winget search vscode	winget search vscode choco search vscode Find-Module PSReadLine	apt search vscode dnf search vscode brew search vscode	
Show package info	winget show Git.Git	winget show Git.Git	apt show git dnf info git brew info git	

Task	CMD	PowerShell	Bash	Notes
Install	winget install Git.Git winget install --id Git.Git -e <i>(exact id)</i>	winget install Git.Git choco install git -y Install-Module PSReadLine <i>(PowerShell module)</i>	sudo apt install git sudo dnf install git brew install git sudo pacman -S git	 Usually needs admin or sudo. brew and per-user winget installs are the exception.
Update	winget upgrade <i>(list available)</i> winget upgrade --all	winget upgrade --all choco upgrade all Update-Module	sudo apt update && sudo apt upgrade sudo dnf upgrade brew update && brew upgrade sudo pacman -Syu	apt update only refreshes the package index. apt upgrade does the installing.
Remove	winget uninstall Git.Git	winget uninstall Git.Git choco uninstall git Uninstall-Module name	sudo apt remove git <i>(keeps config)</i> sudo apt purge git <i>(removes config)</i> sudo apt autoremove <i>(orphaned dependencies)</i> brew uninstall git	

Developer Setup

Task	Windows	macOS	Linux	Notes
Install WSL (Linux on Windows)	wsl --install <i>(Ubuntu by default; reboot afterwards)</i> wsl --install -d Debian <i>(pick a distro)</i> wsl -l -v <i>(list distros and versions)</i> wsl --update wsl --shutdown wsl <i>(open the default distro)</i>	<i>(not needed; macOS is already Unix)</i>	<i>(not needed)</i>	 Run from an administrator prompt. Inside WSL, Windows drives are under /mnt/c and Windows tools are on the PATH, so code . opens VS Code.
Package manager	winget --version <i>(built in on Windows 10 1709+ and 11)</i> winget source update <i>(Chocolatey is optional: see chocolatey.org/install)</i>	/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" brew doctor	<i>(apt, dnf or pacman is already installed)</i> sudo apt update <i>(refresh the index first)</i>	If winget is missing, install "App Installer" from the Microsoft Store.
Git	winget install --id Git.Git -e git --version	xcode-select --install <i>(Apple's git plus compilers)</i> brew install git <i>(newer version)</i>	sudo apt install git sudo dnf install git	On Windows this also installs Git Bash.
GitHub CLI	winget install --id GitHub.cli -e gh auth login	brew install gh gh auth login	sudo apt install gh <i>(Ubuntu 23.04+; older distros: see cli.github.com)</i> gh auth login	gh auth login signs in through the browser and stores credentials that git push uses too.

Task	Windows	macOS	Linux	Notes
SSH key for GitHub	ssh-keygen -t ed25519 -C "you@example.com" Get-Service ssh-agent Set-Service -StartupType Automatic; Start-Service ssh-agent (PowerShell,  admin) ssh-add \$env:USERPROFILE\.ssh\id_ed25519 (PowerShell) gh ssh-key add ~/.ssh/id_ed25519.pub	ssh-keygen -t ed25519 -C "you@example.com" ssh-add --apple-use-keychain ~/.ssh/id_ed25519 gh ssh-key add ~/.ssh/id_ed25519.pub	ssh-keygen -t ed25519 -C "you@example.com" eval "\$(ssh-agent -s)"; ssh-add ~/.ssh/id_ed25519 gh ssh-key add ~/.ssh/id_ed25519.pub	Test with ssh -T git@github.com. The Windows ssh-agent service is off by default.
Node.js	winget install OpenJS.NodeJS.LTS winget install Schniz.fnm (version manager) node -v; npm -v	brew install node brew install fnm (version manager)	curl -fsSL https://fnm.vercel.app/install bash (version manager) fnm install --lts (distro packages are often years old)	A version manager such as fnm or nvm lets you switch Node versions per project.
Python	winget install Python.Python.3.12 python --version py -3 (launcher; picks an installed version)	brew install python python3 --version	sudo apt install python3 python3-pip python3-venv python3 --version	Per-project environment: python -m venv .env, then .env\Scripts\activate (Windows) or source .env/bin/activate.
VS Code	winget install Microsoft.VisualStudioCode code . (open the current folder)	brew install --cask visual-studio-code code . (first run Shell Command: Install code from the Command Palette)	sudo snap install code --classic (or the .deb / .rpm from code.visualstudio.com) code .	
PowerShell 7 and Windows Terminal	winget install Microsoft.PowerShell winget install Microsoft.WindowsTerminal pwsh (start PowerShell 7)	brew install --cask powershell pwsh	(see Microsoft's install page for your distro) pwsh	Windows PowerShell 5.1 is built in. PowerShell 7 (pwsh) is the current cross-platform version and installs alongside it. Windows 11 already includes Windows Terminal.
Compilers and build tools	winget install Microsoft.VisualStudio.2022.BuildTools (choose "Desktop development with C++" in the installer)	xcode-select --install	sudo apt install build-essential sudo dnf groupinstall "Development Tools"	Needed by npm and pip packages that compile native code.
Docker	winget install Docker.DockerDesktop docker run hello-world	brew install --cask docker docker run hello-world	curl -fsSL https://get.docker.com sh sudo usermod -aG docker \$USER (then log out and in) docker run hello-world	Docker Desktop on Windows runs on WSL 2, so install WSL first.
Allow PowerShell scripts	Set-ExecutionPolicy RemoteSigned -Scope CurrentUser (PowerShell) Get-ExecutionPolicy -List	(not needed)	(not needed)	Fixes "running scripts is disabled on this system". RemoteSigned runs local scripts and signed downloads only.

Task	Windows	macOS	Linux	Notes
Check what is installed	<pre>winget list git --version; node -v; python --version; code -v where git <i>(path to the executable)</i></pre>	<pre>brew list git --version; node -v; python3 -- version which git</pre>	<pre>apt list --installed 2>/dev/null grep -i git git --version; node -v; python3 -- version which git</pre>	After installing anything, open a new terminal window. Existing shells keep the old PATH.

Git

Task	Command	Notes
One-time setup	<pre>git config --global user.name "Your Name" git config --global user.email "you@example.com" git config --global init.defaultBranch main git config --list <i>(show all settings)</i></pre>	Settings live in ~/.gitconfig. Drop --global to set them for one repository only.
Start a repository	<pre>git init git clone https://github.com/user/repo.git git clone --depth 1 URL <i>(latest commit only, fast)</i></pre>	
See what changed	<pre>git status git status -s <i>(short form)</i> git diff <i>(unstaged changes)</i> git diff --staged <i>(what will be committed)</i> git diff main..feature <i>(between branches)</i></pre>	
History	<pre>git log --oneline --graph --all git log -p file.txt <i>(changes to one file)</i> git log -5 --stat git show HEAD <i>(last commit in full)</i> git blame file.txt <i>(who changed each line)</i></pre>	
Stage and commit	<pre>git add file.txt git add -A <i>(everything)</i> git add -p <i>(pick hunks interactively)</i> git commit -m "Message" git commit -am "Message" <i>(add tracked files and commit)</i> git commit --amend <i>(fix the last commit)</i></pre>	⚠ Do not amend a commit you have already pushed; it rewrites history others may have.

Task	Command	Notes
Unstage or discard	<pre>git restore --staged file.txt (unstage, keep changes) git restore file.txt (discard local changes) git clean -fd (delete untracked files and folders) git clean -n (dry run first)</pre>	⚠ restore and clean throw work away with no undo. Run git clean -n first.
Branches	<pre>git branch (list) git branch -a (include remote) git switch -c feature (create and switch) git switch main git branch -m old new (rename) git branch -d feature (delete merged) git branch -D feature (force delete)</pre>	switch and restore replaced the overloaded checkout in Git 2.23. checkout still works everywhere.
Merge and rebase	<pre>git merge feature (merge into current branch) git rebase main (replay current branch onto main) git rebase -i HEAD~3 (squash or reorder last 3) git merge --abort / git rebase --abort</pre>	Merge keeps history as it happened. Rebase makes it linear. ⚠ Never rebase commits that are already pushed and shared.
Remotes	<pre>git remote -v git remote add origin https://github.com/user/repo.git git remote set-url origin NEW_URL</pre>	
Push and pull	<pre>git push -u origin main (first push; sets upstream) git push git pull (fetch and merge) git pull --rebase git fetch (download without merging) git push origin --delete feature (delete remote branch)</pre>	
Undo commits	<pre>git revert HEAD (new commit that undoes the last one; safe) git reset --soft HEAD~1 (uncommit, keep changes staged) git reset --mixed HEAD~1 (uncommit, keep changes unstaged) git reset --hard HEAD~1 (uncommit and discard changes) git reflog (find lost commits)</pre>	⚠ reset --hard deletes work. revert is the safe choice for anything already pushed. reflog can recover a reset for about 30 days.
Stash	<pre>git stash (shelve changes) git stash push -m "wip" (with a name) git stash list git stash pop (restore latest and drop it) git stash apply stash@{1} git stash drop</pre>	

Task	Command	Notes
Tags and releases	<code>git tag</code> <i>(list)</i> <code>git tag -a v1.0 -m "Release 1.0"</code> <code>git push --tags</code> <code>git checkout v1.0</code> <i>(look at a tag)</i>	
Ignore and untrack files	<code>git rm --cached file.txt</code> <i>(stop tracking, keep file)</i> <code>git rm -r --cached folder/</code> <code>git check-ignore -v file.txt</code> <i>(why is this ignored?)</i> <code>git mv old.txt new.txt</code> <i>(rename and stage)</i>	Patterns in .gitignore only affect files not yet tracked. Untrack committed files with <code>git rm --cached</code> first.
Search	<code>git grep "pattern"</code> <i>(search tracked files)</i> <code>git log -S "text"</code> <i>(commits that added or removed text)</i> <code>git log --grep "fix"</code> <i>(search commit messages)</i> <code>git log --author="Name"</code>	
Who did what	<code>git shortlog -sn</code> <i>(commits per author)</i> <code>git log --since="2 weeks ago"</code> <code>git bisect start</code> / <code>git bisect good v1.0</code> / <code>git bisect bad</code> <i>(find the breaking commit)</i>	
Help	<code>git help commit</code> <code>git commit -h</code> <i>(short flag list)</i> <code>git <command> --help</code>	

Keyboard Shortcuts

Action	CMD	PowerShell	Bash	Notes
Auto-complete	Tab	Tab <i>(Ctrl+Space shows a menu)</i>	Tab <i>(press twice to list options)</i>	
Previous / next command	↑ / ↓ <i>(F3 repeats the last one)</i>	↑ / ↓	↑ / ↓ or Ctrl+P / Ctrl+N	
Search history	F7 <i>(list)</i> , F8 <i>(cycle prefix matches)</i>	Ctrl+R <i>(backward)</i> , Ctrl+S <i>(forward)</i> , F8	Ctrl+R <i>(backward)</i> , Ctrl+S <i>(forward)</i>	
Clear screen	cls <i>(no shortcut)</i>	Ctrl+L	Ctrl+L	
Cancel running command	Ctrl+C	Ctrl+C	Ctrl+C	
Clear current line	Esc	Esc	Ctrl+U <i>(before cursor)</i> , Ctrl+K <i>(after cursor)</i>	In PowerShell, Ctrl+U and Ctrl+K also work with the default key mode.
Start / end of line	Home / End	Home / End <i>(also Ctrl+A / Ctrl+E)</i>	Ctrl+A / Ctrl+E <i>(Home / End in most terminals)</i>	

Action	CMD	PowerShell	Bash	Notes
Move by word	Ctrl+← / Ctrl+→	Ctrl+← / Ctrl+→	Alt+B / Alt+F (<i>Ctrl+← / → in many terminals</i>)	
Delete word before cursor	<i>(none)</i>	Ctrl+Backspace	Ctrl+W	
Delete word after cursor	<i>(none)</i>	Ctrl+Delete	Alt+D	
Undo an edit	<i>(none)</i>	Ctrl+Z	Ctrl+_ or Ctrl+X Ctrl+U	
Paste	Ctrl+V or right-click	Ctrl+V or right-click	Ctrl+Shift+V (<i>Cmd+V on macOS</i>)	Copy and paste are handled by the terminal app, not the shell. In Bash Ctrl+C cancels and Ctrl+V inserts a literal key.
Copy	Select, then Enter or Ctrl+C	Select, then Enter or Ctrl+C	Ctrl+Shift+C (<i>Cmd+C on macOS</i>)	
Pause / resume output	Ctrl+S / Ctrl+Q	Ctrl+S / Ctrl+Q	Ctrl+S / Ctrl+Q	If the terminal seems frozen, you probably hit Ctrl+S. Press Ctrl+Q.
Suspend foreground job	<i>(none)</i>	<i>(none)</i>	Ctrl+Z (<i>resume with fg or bg</i>)	
Exit shell	exit	exit (<i>Ctrl+D on an empty line</i>)	exit or Ctrl+D	In CMD, Ctrl+Z is the end-of-file character, not exit.

Notes and Gotchas

Paths

- Windows uses backslashes (C:\Users\Alice). Bash uses forward slashes (/home/alice). PowerShell accepts both.
- Quote any path containing spaces: "C:\Program Files" or '/My Files'.
- Windows file systems are case-insensitive. Linux is case-sensitive: File.txt and file.txt are different files. macOS is case-insensitive by default.

Quoting and escaping

- CMD: escape special characters (& < > | ^) with a caret: echo a ^& b. Variables are %NAME%.
- PowerShell: single quotes are literal, double quotes expand \$variables and \$(expressions). The escape character is the backtick (`).
- Bash: single quotes are literal, double quotes expand \$variables and \$(commands). The escape character is the backslash (\).

Wildcards

- * matches any number of characters (*.txt). ? matches exactly one character (file?.txt).
- [abc] and [a-z] match one character from a set (Bash and PowerShell, not CMD).
- In Bash the shell expands wildcards before the command runs. Quote them ("*.txt") when the command should see the pattern itself, as with find and grep.

PowerShell is different

- Commands are Verb-Noun cmdlets (Get-Process, Stop-Service). Aliases such as ls, cat, rm and ps map to cmdlets, but their flags are PowerShell flags: ls -Force, not ls -a.
- Pipelines carry objects, not text. Filter with Where-Object, pick columns with Select-Object, sort with Sort-Object, and format last.
- Windows PowerShell 5.1 ships with Windows. PowerShell 7 (pwsh) is a separate install with && and | |, UTF-8 defaults and Get-Uptime.

macOS, zsh and WSL

- macOS uses zsh as its default shell. Nearly everything in the Bash column works unchanged; profile files are ~/.zshrc instead of ~/.bashrc.
- macOS ships BSD versions of many tools, so some Linux flags differ (sed -i, ls --color, du --max-depth). Commands marked (Linux) do not exist on macOS and vice versa.
- Windows Subsystem for Linux (wsl) gives you a real Linux shell on Windows. Windows drives appear under /mnt/c.

Setting up Windows for development

- Turn on Developer Mode (Settings > System > For developers). It lets you create symlinks without an admin prompt and, on Windows 11 24H2+, enables the built-in sudo.
- Long paths: node_modules trees often exceed the 260-character limit. Run git config --global core.longpaths true, and enable "Enable Win32 long paths" in Group Policy or set LongPathsEnabled=1 under HKLM\SYSTEM\CurrentControlSet\Control\FileSystem (🔒 admin).
- winget installs per user by default; add --scope machine (🔒 admin) to install for everyone.
- Most installers add themselves to the PATH, but only new terminal windows see it. If a command is "not recognized" right after installing, open a new window first.

Git on Windows

- Git for Windows installs Git Bash, a Bash shell where the whole Bash column of this sheet works. Git itself works identically in CMD and PowerShell.
- Line endings: set git config --global core.autocrlf true on Windows (or add a .gitattributes with *text=auto to the repo) so files do not flip between CRLF and LF.
- Credentials: Git for Windows includes Git Credential Manager, which opens a browser sign-in on the first push and remembers it. On macOS the keychain does the same; on Linux use git config --global credential.helper store or the GitHub CLI (gh auth login).
- Lots of "modified" files right after a clone with no real changes usually means a line-ending or file-mode mismatch: git config core.fileMode false and check core.autocrlf.